



2.1. Predicados y Auxiliares

Ejercicio 1. Nombrar los siguientes predicados sobre enteros:

- a) $\text{pred } \text{????}(x : \mathbb{Z}) \{ (\exists c : \mathbb{Z}) (c > 0 \wedge c * c = x) \}$
- b) $\text{pred } \text{????}(x : \mathbb{Z}) \{ \textcolor{red}{x} > 1 \wedge (\forall n : \mathbb{Z}) (1 < n < x \rightarrow_L x \text{ mód } n \neq 0) \}$

Solución

- a) *esCuadradoPerfecto*
- b) *esPrimo*

Extra: Otras maneras de escribir este último predicado.

$\text{pred } \text{esPrimo}(x : \mathbb{Z}) \{ x > 1 \wedge (\forall n : \mathbb{Z}) ((n > 0 \wedge_L x \text{ mód } n = 0) \rightarrow (n = 1 \vee n = x)) \}$

$\text{pred } \text{esPrimo}(x : \mathbb{Z}) \{ x > 1 \wedge \neg(\exists n : \mathbb{Z}) (1 < n < x \wedge_L x \text{ mód } n = 0) \}$

Ejercicio 2. Escriba los siguientes predicados sobre números enteros en lenguaje de especificación:

- a) $\text{pred } \text{divide}(x, y : \mathbb{Z})$ que sea verdadero si y sólo si x divide a y (es decir, el resto es 0).
- b) $\text{pred } \text{sonCoprimos}(x, y : \mathbb{Z})$ que sea verdadero si y sólo si x e y son coprimos.
- c) $\text{pred } \text{mayorPrimoQueDivide}(x : \mathbb{Z}, y : \mathbb{Z})$ que sea verdadero **si y sólo si** y es el mayor primo que divide a x .

Solución

- a) ■ En lenguaje natural, hay un entero k tal que $x \times k = y$.
- Formalmente:
- $\text{pred } \text{divide}(x, y : \mathbb{Z}) \{ (\exists k : \mathbb{Z}) (x \times k = y) \}$
 - $\text{pred } \text{divide}(x, y : \mathbb{Z}) \{ x \neq 0 \wedge_L y \text{ mód } x = 0 \}$
- b) ■ En lenguaje natural, el único divisor común positivo es el 1.
- Formalmente
- $\text{pred } \text{sonCoprimos}(x, y : \mathbb{Z}) \{ (\forall n : \mathbb{Z}) (2 \leq n \leq \min(|x|, |y|) \rightarrow_L \neg(\text{divide}(n, x) \wedge \text{divide}(n, y))) \}$
- c) ■ En lenguaje natural, se debe cumplir que:
- y es primo
 - y divide a x
 - Cualquier otro primo p que divide a x es menor o igual a y
- Formalmente
- $\text{pred } \text{mayorPrimoQueDivide}(x, y : \mathbb{Z}) \{$
- $\text{esPrimo}(y) \wedge_L \text{divide}(y, x) \wedge (\forall p : \mathbb{Z}) ((\text{esPrimo}(p) \wedge_L \text{divide}(p, x)) \rightarrow p \leq y)$
- $\}$

Ejercicio 3. Nombre los siguientes predicados auxiliares sobre secuencias de enteros:

- a) $\text{pred } \text{????}(s : \text{seq}(\mathbb{Z})) \{ (\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] \geq 0) \}$
- b) $\text{pred } \text{????}(s : \text{seq}(\mathbb{Z})) \{ (\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L (\forall j : \mathbb{Z}) (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[i] \neq s[j])) \}$

Solución

- a) *todosPositivos*
- b) *sinRepetidos*

Ejercicio 4. Escriba los siguientes predicados auxiliares sobre secuencias de enteros, aclarando los tipos de los parámetros que recibe:

- a) $\text{pred } \text{pertenece}(\dots)$ que determina si un elemento pertenece a una secuencia.
- b) $\text{pred } \text{esPrefijo}(\dots)$ que determina si una secuencia es prefijo de otra.
- c) $\text{pred } \text{estáOrdenada}(\dots)$ que determina si la secuencia está ordenada de menor a mayor.
- d) $\text{pred } \text{hayUnoParQueDivideAlResto}(\dots)$ que determina si hay un elemento primo en la secuencia que divide a todos los otros elementos de la secuencia.

Solución

- a) $\text{pred } \text{pertenece}(x : \mathbb{Z}, s : \text{seq}(\mathbb{Z})) \{ (\exists i : \mathbb{Z}) (0 \leq i < |s| \wedge_L s[i] = x) \}$
- b) $\text{pred } \text{esPrefijo}(s, t : \text{seq}(\mathbb{Z})) \{ |s| \leq |t| \wedge_L (\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] = t[i]) \}$
- c) $\text{pred } \text{estáOrdenada}(s : \text{seq}(\mathbb{Z})) \{ (\forall i : \mathbb{Z}) (0 \leq i < |s| - 1 \rightarrow_L s[i] \leq s[i + 1]) \}$
- d)
 - En lenguaje natural tenemos que decir que hay *algún elemento de la secuencia* que
 - Es par
 - Divide a todos los elementos de la secuencia
 - Para escribirlo formalmente tenemos dos formas de expresar *algún elemento de la secuencia*
 - Predicando sobre los índices de la secuencia
 $\text{pred } \text{hayUnoParQueDivideAlResto}(s : \text{seq}(\mathbb{Z})) \{ (\exists i : \mathbb{Z}) ((0 \leq i < |s| \wedge_L \text{esPar}(s[i])) \wedge_L (\forall j : \mathbb{Z}) (0 \leq j < |s| \rightarrow_L \text{divideA}(s[i], s[j]))) \}$
 - Predicando sobre los elementos que *pertenecen* a la secuencia
 $\text{pred } \text{hayUnoParQueDivideAlResto}(s : \text{seq}(\mathbb{Z})) \{ (\exists n : \mathbb{Z}) (n \in s \wedge \text{esPar}(n) \wedge_L (\forall m : \mathbb{Z}) (m \in s \rightarrow \text{divideA}(n, m))) \}$

Nota: Para este problema las dos opciones son equivalentes, pero no siempre se puede usar la opción de $\in$.

- Falta definir los predicados auxiliares
 $\text{pred } \text{esPar}(n : \mathbb{Z}) \{ n \bmod 2 = 0 \}$
 $\text{pred } \text{divideA}(n : \mathbb{Z}, m : \mathbb{Z}) \{ n \neq 0 \wedge_L m \bmod n = 0 \}$

Ejercicio 5. Escribir una auxiliar que

- a) Sume 10 a un entero.
- b) “Devuelva” el mayor elemento de una tupla de 3 enteros.

c) “Devuelva” el dígito menos significativo de un entero.

Solución

- a) `aux sumaDiez( $x : \mathbb{Z}$ ) :  $\mathbb{Z} = x + 10$`
- b) `aux mayorDeTupla($t : \text{tupla } \langle \mathbb{Z}, \mathbb{Z}, \mathbb{Z} \rangle$) : $\mathbb{Z} =$
 IfThenElse($t_0 > t_1$, IfThenElse($t_0 > t_2$, t_0 , t_2), IfThenElse($t_1 > t_2$, t_1 , t_2))`
- c) `aux digitoMenosSignificativo( $x : \mathbb{Z}$ ) :  $\mathbb{Z} = x \text{ mód } 10$`

Ejercicio 6. Sea s una secuencia de elementos de tipo $\mathbb{Z}$. Escribir una auxiliar utilizando sumatoria y productoria que:

- a) Cuento la cantidad de veces que aparece el elemento e de tipo $\mathbb{Z}$ en la secuencia s .
- b) Sume los elementos en las posiciones pares de la secuencia s .
- c) Sume los elementos mayores a 0 contenidos en la secuencia s .

Solución

- a) `aux apariciones( $s : \text{seq } \langle \mathbb{Z} \rangle$ ,  $e : \mathbb{Z}$ ) :  $\mathbb{Z} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] = e, 1, 0)$`
- b) `aux sumaPosPares( $s : \text{seq } \langle \mathbb{Z} \rangle$ ) :  $\mathbb{Z} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(i \text{ mód } 2 = 0, s[i], 0)$`
- c) `aux sumaPositivos( $s : \text{seq } \langle \mathbb{Z} \rangle$ ) :  $\mathbb{Z} = \sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] > 0, s[i], 0)$`

2.2. Análisis de especificación

Ejercicio 7. Las siguientes especificaciones no son correctas. Indicar por qué y corregirlas para que describan correctamente el problema.

- a) `proc progresionGeometricaFactor2()` Indica si la secuencia l representa una progresión geométrica factor 2. Es decir, si cada elemento de la secuencia es el doble del elemento anterior.

```
proc progresionGeometricaFactor2(in  $l : \text{seq } \langle \mathbb{Z} \rangle$ ) : bool {  
  requiere { Verdadero }  
  asegura {  $res = \text{True} \leftrightarrow (\forall i : \mathbb{Z}) (0 \leq i < |l| \rightarrow_L l[i] = 2 * l[i - 1])$  }  
}
```

- b) `proc mínimo()` Devuelve en res el menor elemento de l .

```
proc mínimo(in  $l : \text{seq } \langle \mathbb{Z} \rangle$ ) :  $\mathbb{Z}$  {  
  requiere { Verdadero }  
  asegura {  $(\forall y : \mathbb{Z}) ((y \in l \wedge y \neq res) \rightarrow y > res)$  }  
}
```

Solución

a) Está mal definido el rango, cuando $i = 0$, $l[i - 1]$ se indefiniría el consecuente de la implicación.

```
proc progresionGeometricaFactor2(in l : seq(Z)) : bool {
  requiere { Verdadero }
  asegura { res = True  $\leftrightarrow$  ( $\forall i : \mathbb{Z}$ ) ( $1 \leq i < |l| \rightarrow l[i] = 2 * l[i - 1]$ ) }
}
```

b)

- La lista debería tener al menos un elemento.
- El antecedente de la implicación tiene una variable libre innecesaria.
- No garantiza que la salida esté contenida en la secuencia de entrada.

```
proc minimo(in l : seq(Z)) : Z {
  requiere { |l| > 0 }
  asegura { res  $\in l \wedge (\forall y : \mathbb{Z}) (y \in l \rightarrow y \geq res)$  }
}
```

Ejercicio 8. Para los siguientes problemas, dar todas las soluciones posibles a las entradas dadas:

a)

```
proc indiceDelMaximo(in l : seq(R)) : Z {
  requiere { |l| > 0 }
  asegura {  $0 \leq res < |l| \wedge (\forall i : \mathbb{Z}) (0 \leq i < |l| \rightarrow l[i] \leq l[res])$  }
}
```

i) $l = \langle 1, 2, 3, 4 \rangle$

ii) $l = \langle 15.5, -18, 4.215, 15.5, -1 \rangle$

iii) $l = \langle 0, 0, 0, 0, 0, 0 \rangle$

b)

```
proc indiceDelPrimerMaximo(in l : seq(R)) : Z {
  requiere { |l| > 0 }
  asegura {  $0 \leq res < |l| \wedge (\forall i : \mathbb{Z}) (0 \leq i < |l| \rightarrow (l[i] < l[res] \vee (l[i] = l[res] \wedge i \geq res)))$  }
}
```

i) $l = \langle 1, 2, 3, 4 \rangle$

ii) $l = \langle 15.5, -18, 4.215, 15.5, -1 \rangle$

iii) $l = \langle 0, 0, 0, 0, 0, 0 \rangle$

c) ¿Para qué valores de entrada `indiceDelPrimerMaximo` y `indiceDelMaximo` tienen necesariamente la misma salida?

Solución

a) i) $l = \langle 1, 2, 3, 4 \rangle \rightarrow 3$

ii) $l = \langle 15.5, -18, 4.215, 15.5, -1 \rangle \rightarrow \{0, 3\}$

iii) $l = \langle 0, 0, 0, 0, 0, 0 \rangle \rightarrow \{0, 1, 2, 3, 4, 5\}$

b) i) $l = \langle 1, 2, 3, 4 \rangle \rightarrow 3$

ii) $l = \langle 15.5, -18, 4.215, 15.5, -1 \rangle \rightarrow 0$

iii) $l = \langle 0, 0, 0, 0, 0, 0 \rangle \rightarrow 0$

c) Solamente cuando haya una sola aparición del elemento de mayor valor. En los ejemplos, esto ocurre en el caso I.

2.3. Relación de fuerza

Ejercicio 11. Sean x y res variables de tipo $\mathbb{R}$. Considerar los siguientes predicados:

P1: $\{x \leq 0\}$

P2: $\{x \leq 10\}$

P3: $\{x \leq -10\}$

Q1: $\{res \geq x^2\}$

Q2: $\{res \geq 0\}$

Q3: $\{res = x^2\}$

- a) Indicar la relación de fuerza entre P1, P2 y P3
- b) Indicar la relación de fuerza entre Q1, Q2 y Q3
- c) Escribir dos programas que cumplan con la siguiente especificación:

```
proc hagoAlgo(in x :  $\mathbb{R}$ ) :  $\mathbb{R}$  {  
    requiere {  $x \leq 0$  }  
    asegura {  $res \geq x^2$  }  
}
```

- d) Sea A un algoritmo que cumple con la especificación del ítem anterior. Decidir si necesariamente cumple las siguientes especificaciones:

i) requiere $\{x \leq -10\}$, asegura $\{res \geq x^2\}$

ii) requiere $\{x \leq 10\}$, asegura $\{res \geq x^2\}$

iii) requiere $\{x \leq 0\}$, asegura $\{res \geq 0\}$

iv) requiere $\{x \leq 0\}$, asegura $\{res = x^2\}$

v) requiere $\{x \leq -10\}$, asegura $\{res \geq 0\}$

vi) requiere $\{x \leq 10\}$, asegura $\{res = x^2\}$

- e) ¿Qué conclusión pueden sacar? ¿Qué debe cumplirse con respecto a las precondiciones y postcondiciones para que sea seguro **reemplazar la especificación**?

Solución

- a) P3 es más fuerte que P1 y P2. P1 es más fuerte que P2

$$P3 \rightarrow P1, P3 \rightarrow P2, P1 \rightarrow P2$$

- b) Q3 es más fuerte que Q1 y Q2. Q1 es más fuerte que Q2

$$Q3 \rightarrow Q1, Q3 \rightarrow Q2, Q1 \rightarrow Q2$$

- c)

```
func hagaAlgo1(x: float): float {  
    return x * x  
}
```

```
func hagaAlgo2(x: float): float {  
    return x*x + sqrt(-x) - 25*x + 2  
}
```

- d)
 - i) La precondición nueva es más fuerte que la original, por lo tanto alcanza para que ambos algoritmos no se indefinan. La poscondición no cambia.
 - ii) Como la precondición es más débil que la original, no podemos asegurar que el algoritmo vaya a funcionar con los valores de entrada. De hecho, el algoritmo *hagaAlgo2* se indefine cuando recibe valores entre 0 y 10 La poscondición no cambia.
 - iii) La poscondición es más débil que la original, por lo que si nuestros algoritmos cumplían las condiciones originales, deberían cumplir estas también.
 - iv) La poscondición es más fuerte que la original, no tenemos garantía de que se cumpla para los algoritmos propuestos. De hecho, *hagaAlgo2* no cumple con la poscondición propuesta. Si $x = 0$, la salida de la función es $hagaAlgo2(0) = 2$ y se ve que $0 \neq 2$.
 - v) Como la precondición es más fuerte y la poscondición más débil, tenemos garantía que se cumplirán las condiciones dispuestas para cualquier algoritmo que cumpliera con lo pedido en el ítem anterior.

vi) En este caso ocurre todo lo contrario, la precondition es más débil y la postcondition más fuerte, por lo que no tenemos ningún tipo de garantía. Es decir, no sabemos si la salida cumplirá con lo pedido y no tenemos garantía de que nos alcance con los requisitos exigidos sobre la entrada.

e) La **precondición** debería ser más restrictiva, es decir, debería ser más fuerte que la original. Ya que así garantizo que se cumplan los criterios mínimos necesarios para que mi programa no se indefina ni se rompa.

La **poscondición** debe ser más laxa, es decir, debería ser más débil que la original. Ya que así garantizo que todas las salidas posibles cumplen también con el criterio original.

2.4. Especificación de problemas

2.4.1. Problemas sencillos

Ejercicio 13. Especificar los siguientes problemas:

- a) Dado un entero, decidir si es par
- b) Dado un entero n y otro m , decidir si n es un múltiplo de m
- c) Dado un entero, listar todos sus divisores positivos (sin duplicados)
- d) Dado un entero positivo, obtener su descomposición en factores primos. Devolver una secuencia de tuplas (p, e) , donde p es un factor primo y e es su exponente, ordenada en forma creciente con respecto a p

Solución

```
a) proc esPar(in  $n : \mathbb{Z}$ ) : bool {
    requiere { Verdadero }
    asegura {  $res = True \leftrightarrow n \bmod 2 = 0$  }
}

b) proc esMultiploDe(in  $n, m : \mathbb{Z}$ ) : bool {
    requiere {  $m \neq 0$  }
    asegura {  $res = True \leftrightarrow (\exists p : \mathbb{Z}) (n = m \times p)$  }
}

c) proc divisoresPositivos(in  $n : \mathbb{Z}$ ) : seq< $\mathbb{Z}$ > {
    requiere { Verdadero }
    asegura {
         $sinRepetidos(res) \wedge$ 
         $(\forall d : \mathbb{Z}) (d > 0 \wedge_L n \bmod d = 0 \leftrightarrow d \in res)$ 
    }
}

d) proc factoresPrimos(in  $n : \mathbb{Z}$ ) : seq<tupla < $\mathbb{Z}, \mathbb{Z}$ >> {
    requiere {  $n > 0$  }
    asegura {
         $factoriza(n, res) \wedge$ 
         $primerosElementosSonPrimos(res) \wedge$ 
        segundosElementosMayoresAUno(res)  $\wedge$ 
        estaOrdenadaEstrictamentePorPrimerElemento(res)
    }
}
```

```

}

pred factoriza( $n : \mathbb{Z}, s : \text{seq}(\text{tupla } \langle \mathbb{Z}, \mathbb{Z} \rangle)$ ) {  $\prod_{i=0}^{|s|-1} (s[i]_0)^{s[i]_1} = n$  }

pred primerosElementosSonPrimos( $s : \text{seq}(\text{tupla } \langle \mathbb{Z}, \mathbb{Z} \rangle)$ ) {  $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L \text{esPrimo}(s[i]_0))$  }

pred segundosElementosMayoresAUno( $s : \text{seq}(\text{tupla } \langle \mathbb{Z}, \mathbb{Z} \rangle)$ ) {  $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i]_1 > 1)$  }

pred estaOrdenadaEstrictamentePorPrimerElemento( $s : \text{seq}(\text{tupla } \langle \mathbb{Z}, \mathbb{Z} \rangle)$ ) {  $(\forall i : \mathbb{Z}) (0 \leq i < |s| - 1 \rightarrow_L s[i]_0 < s[i+1]_0)$  }

}

```

Ejercicio 14. Dados dos enteros a y b , se necesita calcular su suma y retornarla en un entero c . ¿Cuáles de las siguientes especificaciones son correctas para este problema? Para las que no lo son, indicar por qué.

- a) `proc sumar(inout $a, b, c : \mathbb{Z}$) {
 requiere { Verdadero }
 asegura { $a + b = c$ }
}`
- b) `proc sumar(in $a, b : \mathbb{Z}$, inout $c : \mathbb{Z}$) {
 requiere { Verdadero }
 asegura { $c = a + b$ }
}`
- c) `proc sumar(inout $a, b : \mathbb{Z}$, inout $c : \mathbb{Z}$) {
 requiere { $a = A_0 \wedge b = B_0$ }
 asegura { $a = A_0 \wedge b = B_0 \wedge c = a + b$ }
}`

Solución

- a) **Mal**, se debería especificar que las variables a y b no cambian de valor. Para eso es necesario usar metavariables para poder hablar del estado previo de ambas variables sumando $A_0 = a \wedge B_0 = b$ en el *requiere*. En la cláusula *asegura* se debería pedir $a = A_0 \wedge b = B_0$ y, además, se debería hacer referencia a los valores iniciales de las variables a y b utilizando $c = A_0 + B_0$.
- b) **Bien**, ya que no es necesario restringir la entrada y al hacer referencia a c en la poscondición se predica sobre el valor de salida que debe tener.
- c) **Bien**, es el uso correcto de las metavariables. También se podría especificar el *asegura* como $c = A_0 + B_0$

2.4.2. Problemas sobre secuencias

Ejercicio 15. Especificar los siguientes problemas sobre secuencias:

- a) Dadas dos secuencias s y t , decidir si s está *incluida* en t , es decir, si todos los elementos de s aparecen en t en igual o mayor cantidad
- b) Dadas dos secuencias s y t , devolver su *intersección*, es decir, una secuencia con todos los elementos que aparecen en ambas. Si un mismo elemento tiene repetidos, la secuencia retornada debe contener la cantidad mínima de apariciones del elemento en s y en t .

- c) Dada una secuencia l con todos sus elementos distintos, devolver la secuencia de *partes*, es decir, la secuencia de todas las secuencias incluidas en l , cada una con sus elementos en el mismo orden en que aparecen en l .

Solución

```

a) proc estaIncluida(in  $s, t : \text{seq}(\mathbb{Z})$ ) : bool {
    requiere { Verdadero }
    asegura {  $res = \text{True} \leftrightarrow (\forall e : \mathbb{Z}) (e \in s \rightarrow \text{apariciones}(s, e) \leq \text{apariciones}(t, e))$  }
}

b) proc intersección(in  $s, t : \text{seq}(\mathbb{Z})$ ) :  $\text{seq}(\mathbb{Z})$  {
    requiere { Verdadero }
    asegura {
         $(\forall r : \mathbb{Z}) (r \in res \leftrightarrow r \in s \wedge r \in t) \wedge$ 
         $(\forall e : \mathbb{Z}) (e \in res \rightarrow_L \text{apariciones}(res, e) = \min(\text{apariciones}(s, e), \text{apariciones}(t, e)))$ 
    }
}

c) proc partes(in  $l : \text{seq}(\mathbb{Z})$ ) :  $\text{seq}(\text{seq}(\mathbb{Z}))$  {
    requiere { sinRepetidos( $l$ ) }
    asegura {
         $|res| = 2^{|l|} \wedge$ 
         $(\forall i : \mathbb{Z}) (0 \leq i < |res| \rightarrow (\text{estaIncluida}(res[i], l) \wedge \text{respetarOrden}(res[i], l))) \wedge$ 
         $(\forall i, j : \mathbb{Z}) (0 \leq i, j < |res| \wedge i \neq j \rightarrow \text{sonDistintas}(res[i], res[j]))$ 
    }
    pred estaIncluida( $s, t : \text{seq}(\mathbb{Z})$ ) {  $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow \text{apariciones}(s, s[i]) \leq \text{apariciones}(t, s[i]))$  }
    pred respetarOrden( $s, l : \text{seq}(\mathbb{Z})$ ) {  $(\forall i, j : \mathbb{Z}) (0 \leq i < j < |s| \rightarrow_L$ 
         $(\exists n, m : \mathbb{Z}) (0 \leq n < m < |l| \wedge_L (s[i] = l[n] \wedge s[j] = l[m])))$  }
    pred sonDistintas( $s, t : \text{seq}(\mathbb{Z})$ ) {  $|s| \neq t \vee_L (\exists i : \mathbb{Z}) (0 \leq i < |s| \wedge s[i] \neq t[i])$  }
}

```

Ejercicio 16. Dada una secuencia l , se desea sacar su primer elemento y devolverlo. Decidir cuáles de estas especificaciones son correctas. Para las que no lo son, indicar por qué y justificar con ejemplos.

```

a) proc tomarPrimero(inout  $l : \text{seq}(\mathbb{Z})$ ) :  $\mathbb{Z}$  {
    requiere {  $|l| > 0$  }
    asegura {  $res = \text{head}(l)$  }
}

b) proc tomarPrimero(inout  $l : \text{seq}(\mathbb{Z})$ ) :  $\mathbb{Z}$  {
    requiere {  $|l| > 0 \wedge l = L_0$  }
    asegura {  $res = \text{head}(L_0)$  }
}

c) proc tomarPrimero(inout  $l : \text{seq}(\mathbb{Z})$ ) :  $\mathbb{Z}$  {
    requiere {  $|l| > 0$  }
    asegura {  $res = \text{head}(L_0) \wedge |l| = |L_0| - 1$  }
}

```

```
d) proc tomarPrimero(inout l : seq⟨ℤ⟩) : ℤ {
    requiere { |l| > 0 ∧ l = L0 }
    asegura { res = head(L0) ∧ l = tail(L0) }
}
```

Solución

- a) **Incorrecta.** No define L_0 en la precondition $L_0 = l$, por lo que no pueden indicar que res será el valor del primer elemento de la secuencia a la entrada.
- b) **Incorrecta.** No saca el primero al devolverlo.
- c) **Incorrecta.** No especifica qué elemento saca de la secuencia original.
- d) **Correcta.**

Ejercicio 18. Especificar los siguientes problemas de modificación de secuencias:

- a) **proc reemplazarParesPorSiguiente(inout l : seq⟨ℤ⟩)** que reemplaza los elementos pares de la secuencia por el siguiente. Por ejemplo dada la secuencia $\langle 2, 7, 4 \rangle$ debería devolver $\langle 3, 7, 5 \rangle$
- b) **proc limpiarDuplicados(inout l : seq⟨char⟩) : seq⟨char⟩**, que elimina los elementos duplicados de l dejando sólo su primera aparición (en el orden original). Devuelve además una secuencia con todas las apariciones eliminadas (en cualquier orden)

Solución

```
a) proc reemplazarParesPorSiguiente(inout l : seq⟨ℤ⟩) {
    requiere { l = L0 }
    asegura {
        |l| = |L0| ∧L
        (∀i : ℤ) (0 ≤ i < |l| →L (esPar(L0[i]) → l[i] = L0[i] + 1) ∧ (¬esPar(L0[i]) → l[i] = L0[i]))
    }
}

b) proc limpiarDuplicados(inout l : seq⟨char⟩) : seq⟨char⟩ {
    requiere { l = L0 }
    asegura {
        sinRepetidos(l) ∧
        mismosElementos(l, L0) ∧
        mismasAparicionesTotales(l, res, L0) ∧
        mismoOrden(l, L0)
    }
}

pred sinRepetidos(s : seq⟨char⟩) { (∀c : char) (c ∈ s →L apariciones(s, c) ≤ 1) }
pred mismosElementos(s, t : seq⟨char⟩) { (∀c : char) (c ∈ s ↔ c ∈ t) }
pred mismasAparicionesTotales(s, t, u : seq⟨char⟩) { (∀c : char) (apariciones(s, c) + apariciones(t, c) = apariciones(u, c)) }
pred mismoOrden(s, t : seq⟨char⟩) { (∀i, j : ℤ) (0 ≤ i < j < |s| →L (∃k, m : ℤ) ((0 ≤ k < m < |t|) ∧L (s[i] = t[k] ∧ s[j] = t[m] ∧ esPrimeraAparicion(s[i], t, k) ∧ esPrimeraAparicion(s[j], t, m)))) }
pred esPrimeraAparicion(c : char, s : seq⟨char⟩, k : ℤ) { s[k] = c ∧ (∀j : ℤ) (0 ≤ j < k →L s[j] ≠ c) }
```

2.4.3. Problemas sobre conjuntos y diccionarios

Ejercicio 19. Dado un conjunto de enteros, se espera que el siguiente predicado sea Verdadero cuando todos sus elementos son mayores a 0. Indicar por qué son incorrectas las siguientes especificaciones y proponer una valida.

- a) `pred sonTodosPositivos(c : conj⟨ℤ⟩) { (∀i : ℤ) (0 ≤ i < |c| → c[i] > 0) }`
- b) `pred sonTodosPositivos(c : conj⟨ℤ⟩) { (∀e : ℤ) (e ∈ c ↔ e > 0) }`
- c) `pred sonTodosPositivos(c : conj⟨ℤ⟩) { res = True ↔ (∀e : ℤ) (e ∈ c → e > 0) }`

Solución

- a) La forma de predicar sobre un conjunto no es igual a la que tenemos para predicar sobre una secuencia, no hay índices, pues no hay una noción de orden en los elementos en el conjunto.
- b) Al haber un "si y solo si", este predicado dará verdadero solo si todos los enteros positivos están en el conjunto, lo cual no buscamos.
- c) Un predicado no tiene un *res*, este directamente devuelve un valor de verdad.

Predicado correcto: `pred sonTodosPositivos(c : conj⟨ℤ⟩) { (∀e : ℤ) (e ∈ c → e > 0) }`

Ejercicio 20. Especificar los siguientes procedimientos:

- a) `proc sacarImpares(inout c : conj⟨ℤ⟩)` Que saca los elementos impares del conjunto *c*
- b) `proc devolverYEliminarMasChico(inout c : conj⟨ℤ⟩) : ℤ` Recibe un conjunto no vacío, elimina el menor elemento de éste y lo devuelve

Solución

- a)

```
proc sacarImpares(inout c : conj⟨ℤ⟩) {  
  requiere { c = C0 }  
  asegura {  
    (∀e : ℤ) (e ∈ c ↔ (e ∈ C0 ∧ esPar(e)))  
  }  
}
```
- b)

```
proc devolverYEliminarMasChico(inout c : conj⟨ℤ⟩) : ℤ {  
  requiere { c = C0 ∧ |c| > 0 }  
  asegura {  
    (∃m : ℤ) (m ∈ C0 ∧ (∀e : ℤ) (e ∈ C0 → m ≤ e) ∧ c = C0 - {m} ∧ res = m)  
  }  
}
```

Ejercicio 21. Dado un diccionario, se espera que el siguiente predicado sea Verdadero cuando no hay valores iguales para distintas claves. Indicar por qué son incorrectas las siguientes especificaciones y proponer una valida.

- a) `pred noHayValoresRepetidos(d : dicc⟨K, V⟩) { (∀c1 : K) (c1 ∈ d ∧L (∀c2 : K) (d[c1] ≠ d[c2])) }`
- b) `pred noHayValoresRepetidos(d : dicc⟨K, V⟩) { (∀c1 : K) (c1 ∈ d →L (∀c2 : K) (c2 ∈ d →L c1 ≠ c2)) }`
- c) `pred noHayValoresRepetidos(d : dicc⟨K, V⟩) { (∀c1 : K) (c1 ∈ d →L (∀c2 : K) (c2 ∈ d →L d[c1] ≠ d[c2])) }`

Solución

- a) Este predicado dará siempre Falso independientemente de la entrada, ya que estamos utilizando un $\wedge_L$ en el $\forall$, y claramente habrá una c_1 que no este en d , quedandonos $\dots \wedge \text{false} \wedge \dots$.
- b) Está diciendo que las claves deben ser distintas, el enunciado pide decir que los valores asociados a claves distintas siempre sean diferentes. Aparte, este predicado siempre dará Falso para un diccionario no vacío, ya que en alguno de esos $\wedge$ implícitos en el $\forall$ va a haber un $c_1 = c_1$, pues estamos predicando para todas las claves en ambos.
- c) Acá, al igual que arriba, va a haber un caso donde $c_1 = c_2$, lo cual ocasionará que termine dando Falso.

Predicado correcto:

```
pred noHayValoresRepetidos( $d : \text{dicc}\langle K, V \rangle$ ) {
  ( $\forall c_1 : K$ ) ( $c_1 \in d \rightarrow_L (\forall c_2 : K) ((c_2 \in d \wedge c_1 \neq c_2) \rightarrow_L d[c_1] \neq d[c_2])$ )
}
```

Ejercicio 22. Dado el siguiente procedimiento:

```
proc valorMasGrande(in  $d : \text{dicc}\langle \mathbb{Z}, \mathbb{Z} \rangle$ ) :  $\mathbb{Z}$ 
```

- a) Especificarlo, considerando que recibe un diccionario con tanto claves como valores números positivos, y da como resultado el valor más grande asociado a alguna clave del diccionario.
- b) ¿La solución propuesta permite que la entrada sea un diccionario vacío? Modifíquela para que, en dicho caso, devuelva -1.

Solución

```
a) proc valorMasGrande(in  $d : \text{dicc}\langle \mathbb{Z}, \mathbb{Z} \rangle$ ) :  $\mathbb{Z}$  {
  requiere {  $|d| > 0$  }
  asegura {
    ( $\exists c : \mathbb{Z}$ ) ( $c \in d \wedge_L (\forall k : \mathbb{Z}) (k \in d \rightarrow_L d[k] \leq d[c]) \wedge res = d[c]$ )
  }
}

b) proc valorMasGrande(in  $d : \text{dicc}\langle \mathbb{Z}, \mathbb{Z} \rangle$ ) :  $\mathbb{Z}$  {
  asegura {
    ( $|d| > 0 \rightarrow (\exists c : \mathbb{Z}) (c \in d \wedge_L (\forall k : \mathbb{Z}) (k \in d \rightarrow_L d[k] \leq d[c]) \wedge res = d[c])$ )  $\wedge$ 
    ( $|d| = 0 \rightarrow res = -1$ )
  }
}
```

Ejercicio 23. En nuestro lenguaje de especificación contamos con dos funciones útiles llamadas setKey y delKey, las cuales funcionan como detalla el apunte. Suponer que no existen dichas funciones y especificar los siguientes procedimientos:

- a) proc eliminarClave(inout $d : \text{dicc}\langle K, V \rangle$, in $c : K$) Que hace $d = \text{delKey}(D_0, c)$
- b) proc agregarOModificarSiExiste(inout $d : \text{dicc}\langle K, V \rangle$, in $c : K$, in $val : V$) Que hace $d = \text{setKey}(D_0, c, e)$

Solución

```
a) proc eliminarClave(inout  $d : \text{dicc}\langle K, V \rangle$ , in  $c : K$ ) {
  requiere {  $d = D_0$  }
```

```

    asegura {
       $(\forall e : K) (e \in d \leftrightarrow (e \in D_0 \wedge e \neq c)) \wedge_L$ 
       $(\forall e : K) ((e \in D_0 \wedge e \neq c) \rightarrow_L d[e] = D_0[e])$ 
    }
  }

```

b) `proc agregarOModificarSiExiste`(inout $d : \text{dicc}\langle K, V \rangle$, in $c : K$, in $val : V$) {
 requiere { $d = D_0$ }
 asegura {
 $\text{ExisteClaveValor}(d, c, val) \wedge$
 $\text{ClavesViejasSalvoQuizasEnUna}(d, D_0, c) \wedge$
 $\text{ValoresViejoSalvoQuizasEnUno}(d, D_0, c)$
 }
 asegura {
 $(c \in d \wedge_L d[c] = val) \wedge$
 $(\forall k : K) (k \neq c \rightarrow_L k \in d \leftrightarrow k \in D_0) \wedge$
 $(\forall k : K) ((k \neq c \wedge k \in d \wedge k \in D_0) \rightarrow_L d[k] = D_0[k])$
 }
}

Ejercicio 24. Dado el siguiente procedimiento:

`proc modificarMayores`(inout $d : \text{dicc}\langle \mathbb{Z}, \mathbb{Z} \rangle$, in $umbral : \mathbb{Z}$, in $v : \mathbb{Z}$)

que cambia el valor de todas las claves de d que superen el umbral por el valor v , dejando las otras como estan, por ejemplo:

```

Umbral = 10, v = 100
D0 = {3 → 2, 6 → 12, 20 → 0, 9 → 1, 32 → 5, 10 → -2}
d = {3 → 2, 6 → 12, 20 → 100, 9 → 1, 32 → 100, 10 → -2}

```

- Especificar el problema
- Para este problema es incorrecto utilizar `setKey`. Explique por qué y corrija su especificación en caso de haberlo usado.
- ¿Qué habría que agregar para que, a parte de modificar d , devuelva el conjunto de todas las claves cuyos valores fueron modificados? Es decir, $\{20, 32\}$ en el ejemplo dado.

Solución

```

a) proc modificarMayores(inout  $d : \text{dicc}\langle \mathbb{Z}, \mathbb{Z} \rangle$ , in  $umbral : \mathbb{Z}$ , in  $v : \mathbb{Z}$ ) {
  requiere {  $d = D_0$  }
  asegura {
     $(\forall c : \mathbb{Z}) (c \in d \leftrightarrow c \in D_0) \wedge_L ($ 
       $(\forall c : \mathbb{Z}) ((c \in D_0 \wedge c \leq umbral) \rightarrow_L d[c] = D_0[c]) \wedge$ 
       $(\forall c : \mathbb{Z}) ((c \in D_0 \wedge c > umbral) \rightarrow_L d[c] = v)$ 
    )
  }
}

```

- b) *setKey* cambia el valor de una clave por otro (o crea una clave con cierto valor) en un diccionario, sin modificar el resto de las claves. Una opción muy tentadora, pero totalmente **incorrecta** es hacer un $(\forall c : \mathbb{Z}) (c \in D_0 \rightarrow_L d = \text{setKey}(D_0, c, \text{ifThenElse}(c \leq \text{umbral}, c, v)))$, pero esto es algo que está muy **mal**, ya que el $\forall$ es una concatenación de $\wedge$, por lo que tendríamos muchos diccionarios d igualados a cosas distintas separados por $\wedge$, lo cual haría que sea siempre falso. Este error es muy común al pensar que especificar es igual a programar, creyendo que es similar a un ciclo que irá modificando los elementos, cuando no es así.
- c) Habría que agregar en el asegura: $\wedge (\forall c : \mathbb{Z}) (c \in \text{res} \leftrightarrow (c \in D_0 \wedge c > \text{umbral}))$

2.5. Ejercicios de parciales anteriores

Ejercicio 25. Se desea especificar el problema **reemplazarNúmerosPerfectos**, que dada una secuencia de enteros devuelve la secuencia pero con los valores que se corresponden con números perfectos reemplazados por el índice donde se encuentran. Se llama números perfectos a aquellos naturales mayores a cero que son iguales a la suma de sus divisores positivos propios (divisores incluyendo al 1 y sin incluir al propio número). Por ejemplo, *reemplazarNúmerosPerfectos*([0, 3, 9, 6, 4, 28, 7]) = [0, 3, 9, 3, 4, 5, 7], donde los únicos números reemplazados son el 6 y el 28 porque son los únicos números perfectos de la secuencia.

Solución

```

proc reemplazarNumerosPerfectos(in l : seq<Z>) : seq<Z> {
  requiere { Verdadero }
  asegura {
    |l| = |res|  $\wedge_L$  (
      perfectosReemplazados(l, res)  $\wedge$ 
      noPerfectosIguales(l, res)
    )
  }
  pred perfectosReemplazados(l : seq<Z>, res : seq<Z>) {  $(\forall i : \mathbb{Z}) ((0 \leq i < |l| \wedge_L \text{esPerfecto}(l[i])) \rightarrow_L \text{res}[i] = i)$  }
  pred noPerfectosIguales(l : seq<Z>, res : seq<Z>) {  $(\forall i : \mathbb{Z}) ((0 \leq i < |l| \wedge_L \neg \text{esPerfecto}(l[i])) \rightarrow_L \text{res}[i] = l[i])$  }
  pred esPerfecto(n : Z) {  $n > 0 \wedge n = \sum_{i=1}^{n-1} \text{IfThenElse}(\text{esDivisor}(n, i), i, 0)$  }
  pred esDivisor(n : Z, i : Z) {  $n \bmod i = 0$  }
}

```

Ejercicio 26. Se desea especificar el problema **ordenarYBuscarMayor** que dada una secuencia s de enteros (que puede tener repetidos) ordena dicha secuencia en orden creciente de valor absoluto y devuelve el valor del máximo elemento. Por ejemplo,

- *ordenarYBuscarMayor*([1, 4, 3, 5, 6, 2, 7]) = [1, 2, 3, 4, 5, 6, 7], 7
- *ordenarYBuscarMayor*([1, -2, 2, 5, 1, 4, -2, -10]) = [1, 1, -2, -2, 2, 4, 5, -10], 5
- *ordenarYBuscarMayor*([-10, -3, -7, -9]) = [-3, -7, -9, -10], -3

Solución

```

proc ordenarYBuscarMayor(inout s : seq<Z>) : Z {
  requiere {  $s = S_0$  }
  asegura {
    mismosElementos(s, S0)  $\wedge_L$  (
      estaOrdenadaPorValorAbs(s)  $\wedge$ 
      esMaximo(s, res)
    )
  }
  pred mismosElementos(s : seq<Z>, t : seq<Z>) {  $(\forall e : \mathbb{Z}) (\text{apariciones}(s, e) = \text{apariciones}(t, e))$  }
  pred apariciones(s : seq<Z>, e : Z) {  $\sum_{i=0}^{|s|-1} \text{IfThenElse}(s[i] = e, 1, 0)$  }
  pred estaOrdenadaPorValorAbs(s : seq<Z>) {  $(\forall i : \mathbb{Z}) (0 \leq i < |s| - 1 \rightarrow_L |s[i]| \leq |s[i+1]|)$  }
  pred esMaximo(s : seq<Z>, m : Z) {  $(\exists i : \mathbb{Z}) (0 \leq i < |s| \wedge_L s[i] = m) \wedge (\forall j : \mathbb{Z}) (0 \leq j < |s| \rightarrow_L s[j] \leq m)$  }
}

```

}

Ejercicio 27. Se desea especificar el problema `procesarPrefijos` que dada una secuencia s de palabras y una palabra p , remueve todas las palabras de s que no tengan como prefijo a p y además retorna la longitud de la palabra más larga que tiene de prefijo a p . Por ejemplo, dados: $s = ["casa", "calamar", "banco", "recuperatorio", "aprobar", "cansado"]$ y $p = "ca"$ un posible valor para la secuencia s luego de aplicar `procesarPrefijos(s, p)` puede ser $["casa", "calamar", "cansado"]$ y el valor devuelto será 7.

Solución

```
proc procesarPrefijos(inout s : seq<string>, in p : string) : ℤ {
  requiere { s = S0 }
  asegura {
    (∀ r : string) (r ∈ s ↔ (r ∈ S0 ∧ esPrefijo(p, r))) ∧
    maxTamañoConPrefijo(S0, p, res)
  }
  pred maxTamañoConPrefijo(s : seq<string>, p : string, m : ℤ) {
    (∃ r : string) (r ∈ s ∧ esPrefijo(p, r) ∧ |r| = m) ∧
    (∀ r : string) ((r ∈ s ∧ esPrefijo(p, r)) → |r| ≤ m)
  }
}
```